Model Optimisation for Edge Deployment

Quantisation (PTQ, QAT) and Pruning (unstructured, structured) of a CIFAR-10 ResNet-18

Assignment 03 — Edge Computing & Intelligent Systems

Platform: Apple M4 (macOS), PyTorch 2.8, Python 3.9, ONNX-Runtime 1.19. All CPU timings use 1 thread unless stated.

1. Summary of results

All five models were evaluated on the full 10,000-image CIFAR-10 test set with one automated script (benchmark.py). Sizes are the state-dict checkpoints on disk; latency is batch-1 on one CPU thread; throughput is batch-128.

Model	Acc %	Drop (pts)	Size MB	Gzip MB	Params	Sparsity %	MACs (M)	Lat. bs1 (ms)	Img/s bs128	Speed-up
FP32 baseline	93.43	0.0	44.77	41.49	11173962	0.0	555.4	5.253	175.8	1.0
PTQ INT8	93.53	-0.1	11.31	9.97	11173962	0.0	555.4	4.395	242.8	1.2
QAT INT8	93.24	0.19	11.22	8.83	11173962	0.0	555.4	4.361	251.4	1.2
Unstructured pruned	93.47	-0.04	44.77	7.09	11173962	90.0	555.4	5.45	174.8	0.96
Structured pruned	93.34	0.09	15.72	14.56	3913470	0.0	293.2	3.524	303.6	1.49

- PTQ-INT8: 3.96x smaller, 1.20x faster at batch 1, accuracy 93.53% (FP32 93.43%).
- QAT-INT8: 3.99x smaller, 1.20x faster, accuracy 93.24%.
- Unstructured pruning (90% sparse): accuracy 93.47%, gzip size 5.8x smaller, but NO speed-up on a dense CPU runtime (0.96x).
- Structured pruning: 65% fewer parameters, 47% fewer MACs, 1.49x faster, accuracy 93.34%.

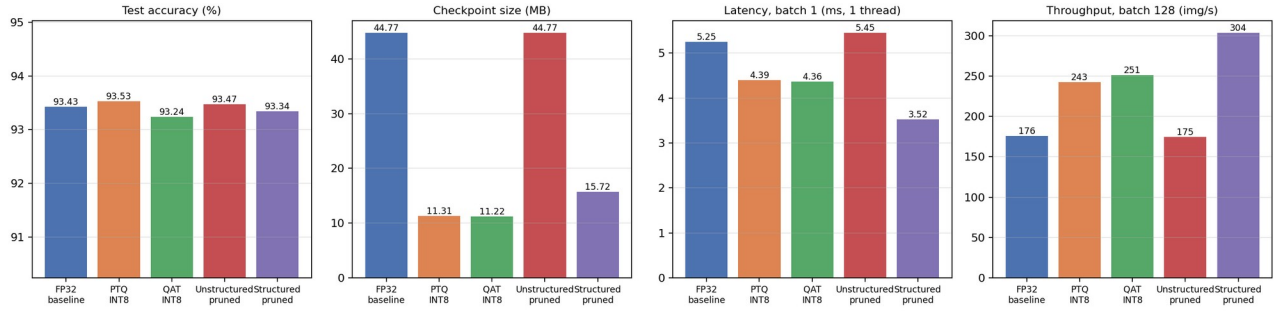


Figure 1 - accuracy, size, latency and throughput of the five models.

Statistical caveat: with 10,000 test images the binomial standard error of an accuracy near 93% is about 0.25 percentage points, and every result is a single run with a fixed seed. Differences of a few tenths of a point between the five models (including PTQ appearing 0.10 pts 'better' than FP32) are therefore within noise and should be read as 'no measurable accuracy loss', not as one method being better.

2. Implementation

2.1 Common infrastructure (common.py)

- Data: CIFAR-10 held in memory as uint8; per-batch normalisation and random-crop (4 px padding) + horizontal-flip augmentation done on the accelerator. The test set is only used for evaluation, never for training or calibration.
- Model: CIFAR-style ResNet-18 (3x3 stem, no max-pool, 11.17 M parameters, 555 M MACs). Every residual add+ReLU uses FloatFunctional.add_relu so the network can be quantised in eager mode; the block constructor takes the inner width as an argument so a physically smaller network can be rebuilt after channel pruning.
- Reproducibility: fixed seed 42, run_all.sh runs the whole pipeline, every step logs to logs/.

2.2 Baseline FP32 (train_baseline.py)

20 epochs, SGD (Nesterov momentum 0.9, weight-decay 5e-4), one-cycle LR schedule with peak 0.1, batch 128, on the Apple GPU (MPS). The final-epoch model is saved (no best-epoch selection on the test set). Result: 93.43% test accuracy (training took 53 minutes).

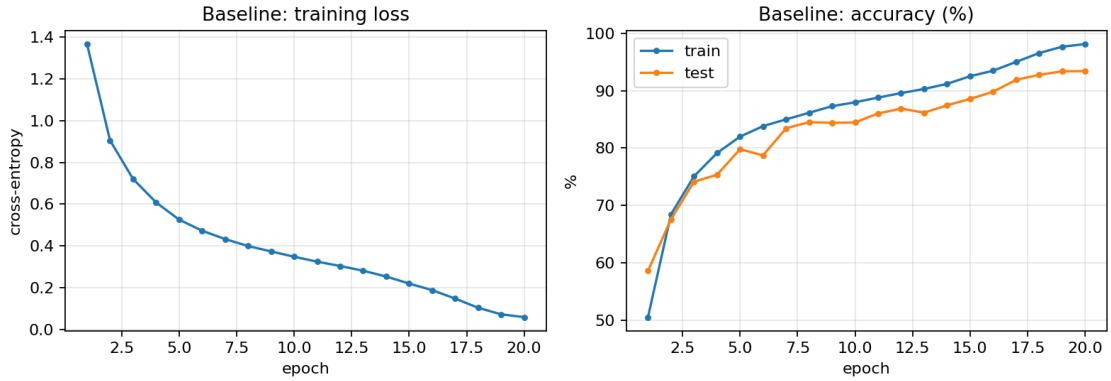


Figure 2 - baseline training curves.

2.3 Post-training static quantisation (ptq.py, calibration.py)

- Fusion: Conv+BN+ReLU, Conv+BN (including the 1x1 shortcut convolutions) are fused with fuse_modules (eval mode).
- Backend: torch.backends.quantized.engine is chosen at run time - FBGEMM on x86, QNNPACK on this ARM machine (FBGEMM is not available on Apple silicon).
- QConfig: per-channel symmetric INT8 weights, UINT8 activations with a HistogramObserver.
- Calibration: 2,000 randomly chosen TRAINING images (no augmentation) are passed through the observed model.
- Convert + save: converted to INT8, evaluated, saved both as state_dict and as self-contained TorchScript; the script reloads the TorchScript file and checks that it reproduces the in-memory accuracy exactly.

Result: 93.53% test accuracy, 11.31 MB (3.96x smaller).

2.4 Quantisation-aware training (qat.py)

- Initialised from the pretrained FP32 checkpoint; Conv+BN+ReLU fused with the QAT fusion (train mode); FakeQuantize modules inserted with prepare_qat (QNNPACK QAT qconfig).

- Fine-tuning: 3 epochs x 20,000 random training images, SGD lr 0.005 with cosine decay. BatchNorm statistics are frozen from epoch 2 and the observers from epoch 3. QAT runs on the CPU because the fused fake-quant operator is not implemented for the Apple GPU (MPS); this limited the budget to ~15 minutes per epoch.
- Convert to INT8, evaluate, save state_dict + TorchScript (+ the fake-quant model for reference).

Result: 93.24% test accuracy, 11.22 MB. In this experiment QAT did not beat PTQ: the FP32 model is quantisation-friendly (per-channel weights + histogram calibration already lose nothing measurable), so there is little for QAT to recover, and QAT used per-tensor weights and a very short schedule. We report this as measured rather than tune toward the textbook expectation.

2.5 Unstructured pruning (prune_unstructured.py)

- Global L1-magnitude pruning over all Conv2d and Linear weights (torch.nn.utils.prune), with masks (weight_orig + weight_mask) active during fine-tuning.
- Iterative: 5 pruning iterations on a cubic sparsity schedule (43.9 -> 70.6 -> 84.2 -> 89.3 -> 90.0 %), one fine-tuning epoch after each, then 6 final fine-tuning epochs.
- Permanent: prune.remove bakes the mask into the weight tensors; the script checks that sparsity (90.00%) and accuracy are unchanged by this step.

Observation: global pruning removes far more from the wide, redundant last stage (layer4 convolutions 93-99.9% sparse) than from the early layers (40-45%), and the first conv and fully-connected layer are barely pruned.

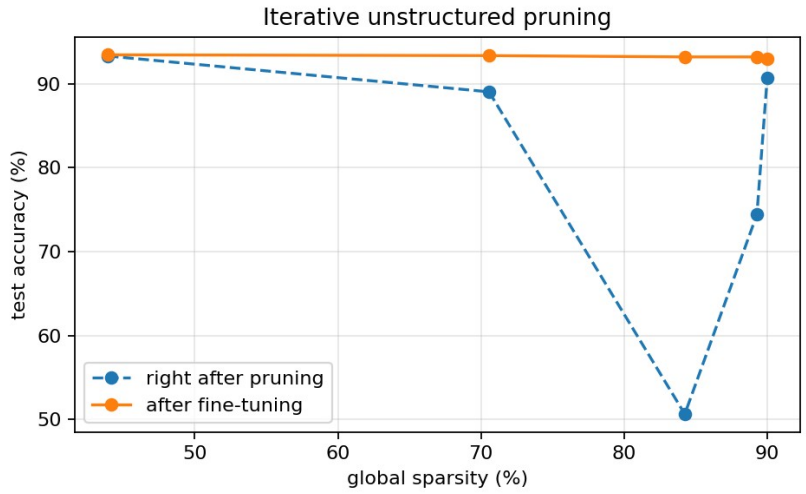


Figure 3 - accuracy right after each pruning step and after fine-tuning.

2.6 Structured channel pruning (prune_structured.py)

- Scope: the inner width of each of the 8 residual blocks (conv1 output = conv2 input). The residual-stream width is tied to the shortcut connections and the next block's input, so leaving it untouched keeps every addition dimensionally valid without touching other layers.
- Ranking: L1 norm of each conv1 filter. Pruned fraction per block: 25/25/40/40/60/60/70/70 % (more in the later, wider stages).
- Reconstruction: a new, physically smaller ResNet-18 is instantiated with the reduced widths [48, 48, 77, 77, 102, 102, 154, 154] and the surviving weights (conv1 filters, bn1 channels, conv2 input channels) are copied in. No masks and no zero-valued channels remain.

- Verification: the rebuilt network's logits equal those of the original network with the pruned channels masked to zero (max difference 0 on a test batch). The script asserts this before fine-tuning.
- Fine-tuning: 12 epochs, SGD lr 0.02 with cosine decay. Accuracy fell to 30.2% straight after pruning and recovered to 93.34%.

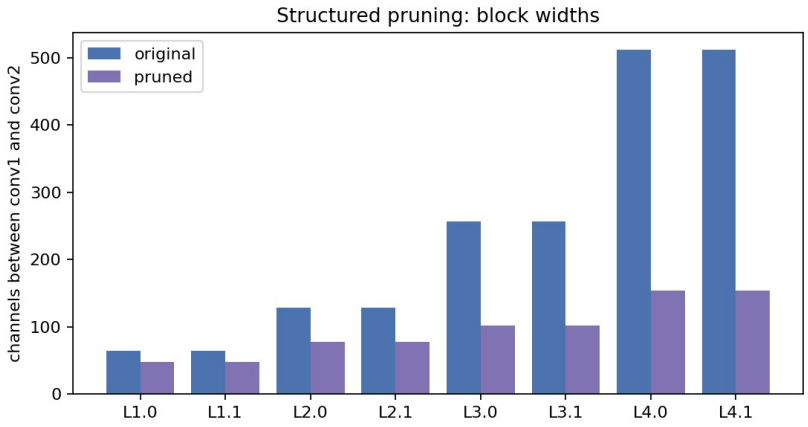


Figure 4 - internal block widths before / after structured pruning.

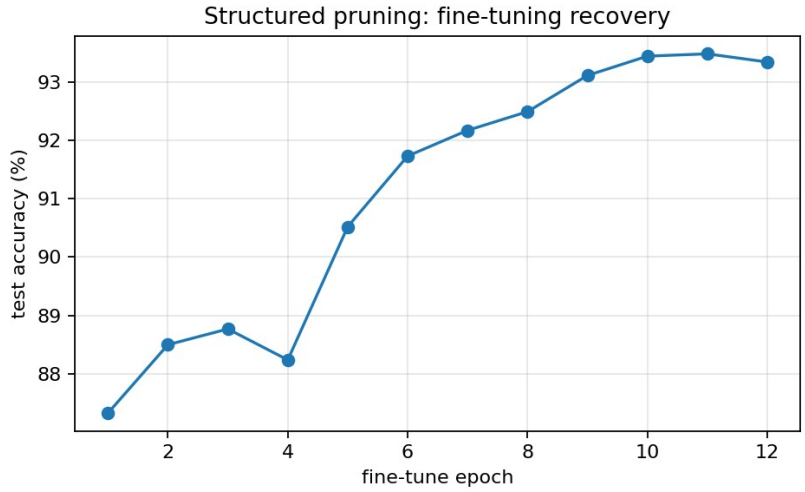


Figure 5 - recovery of the structured-pruned model during fine-tuning.

2.7 Inference pipeline (inference.py)

A single entry point loads any of the five models (FP32 / PTQ-INT8 / QAT-INT8 / unstructured / structured) through load_model(key): float models are rebuilt from their checkpoint (the structured checkpoint stores its channel widths), INT8 models are loaded from TorchScript with the right quantisation backend selected first. It classifies CIFAR-10 test samples or arbitrary image files (resized to 32x32) and prints a side-by-side table.

```
$ python inference_demo.py   [Inference demo]

input truth | FP32 baseline | PTQ INT8 | QAT INT8 | Unstructured pruned | Structured pruned
...
test[6044] horse | horse 99.9% ok | horse 99.9% ok | horse 100.0% ok | horse 99.9% ok | horse 99.9% ok
test[2890] dog | dog 70.8% ok | dog 69.4% ok | dog 46.2% ok | dog 98.3% ok | dog 98.4% ok
test[9399] horse | horse 100.0% ok | horse 100.0% ok | horse 100.0% ok | horse 100.0% ok | horse 100.0% ok
test[1917] ship | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok
test[4919] ship | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok
test[1729] ship | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok | ship 100.0% ok
test[429] frog | frog 100.0% ok | frog 100.0% ok | frog 100.0% ok | frog 100.0% ok | frog 100.0% ok
test[7532] automobile | automobile 99.9% ok | automobile 99.9% ok | automobile 100.0% ok | automobile 100.0% ok | automobile 100.0% ok
```

Figure 6 - inference.py on 8 random test images, all five models.

2.8 Benchmark (benchmark.py)

Evaluates all five models on the full test set and writes: results/benchmark_results.csv (summary), results/predictions.csv (per-image label, prediction and confidence for each model), and the per-question subsets q1_quantization_results.csv, q1_quantization_predictions.csv, q2_pruning_results.csv. Latency is the mean of 200 batch-1 runs after 30 warm-up runs (p50/p95 also recorded); throughput is measured over 768 test images at batch 128.



Figure 7 - benchmark.py console output.

3. ONNX deployment

export_onnx.py exports all five models; onnx_inference.py runs them with ONNX-Runtime's CPU provider (no PyTorch needed for inference) and writes results/onnx_benchmark.csv.

- FP32, unstructured and structured models: direct torch.onnx.export (opset 13, dynamic batch).
- INT8 models: PyTorch eager-mode quantised operators have no ONNX representation. The converted INT8 model is therefore re-expressed as a QDQ graph: the exact INT8 weights, per-channel scales, activation scales and zero-points are read from the converted PyTorch model and written as QuantizeLinear/DequantizeLinear nodes around Conv/Gemm/Add/ReLU, then weight Quantize nodes are constant-folded to real INT8 initialisers. The ONNX INT8 models therefore carry the PTQ-calibrated / QAT-learned parameters (not a re-calibration).
- Fidelity: the QDQ twin agrees with the PyTorch INT8 model on 99.6% (PTQ) / 99.8% (QAT) of 500 test images; small differences come from the float emulation of integer arithmetic.

ONNX model	Acc %	Agreement w/ FP32 %	Size MB	Lat. bs1 (ms)	Img/s	Speed-up
FP32 baseline	93.43	100.0	44.69	11.29	92.7	1.0
PTQ INT8 (QDQ)	93.42	99.66	11.24	12.048	91.1	0.94
QAT INT8 (QDQ)	93.27	97.88	11.22	12.016	90.2	0.94
Unstructured pruned	93.47	96.78	44.69	11.281	92.8	1.0
Structured pruned	93.34	95.53	15.65	6.14	166.4	1.84

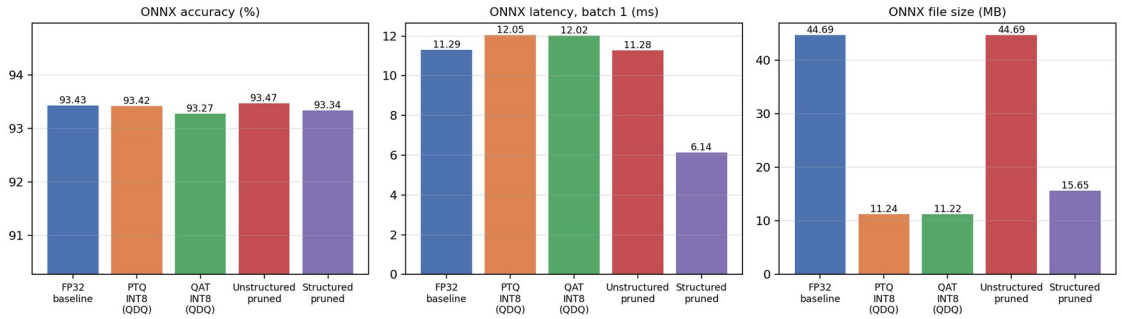


Figure 8 - ONNX-Runtime CPU results.

Honest finding: ONNX accuracies match PyTorch (93.43% / 93.42% / 93.27% / 93.47% / 93.34%). The INT8 ONNX files are 4.0x smaller but are NOT faster under ONNX-Runtime on this Apple CPU (0.94x), unlike PyTorch's QNNPACK kernels (1.2x). ONNX-Runtime's FP32 convolutions are well optimised for this CPU and its QDQ execution adds quantise/dequantise overhead; INT8 speed-ups depend strongly on the target CPU and runtime. The structured-pruned model gives the largest real speed-up (1.84x) because it simply has fewer operations.

4. Discussion

- Quantisation (PTQ/QAT) gives a reliable ~4x reduction in model size with no measurable accuracy change, and a modest 1.2x speed-up on this CPU in PyTorch. Calibration with 2,000 images was enough; PTQ took about a minute.
- Unstructured pruning reaches 90% sparsity with no accuracy loss, but the zeros live inside dense tensors: the file is the same size and the dense kernels do the same work, so latency does not improve. The benefit shows up only with compression (gzip: 41.5 -> 7.1 MB) or a sparse-aware runtime/hardware.
- Structured pruning delivers genuine compute savings with standard dense kernels (-65% parameters, -47% MACs, 1.5x faster in PyTorch, 1.8x in ONNX-Runtime), at the cost of a larger accuracy hit directly after pruning (30%) that needs a proper fine-tuning budget; after 12 epochs it is within 0.1 pts of the baseline.
- The techniques are complementary (e.g. structured pruning followed by INT8 quantisation) but combining them was outside the assignment's scope and was not evaluated.

5. Limitations

- Single seed, single run per method; the test set doubled as the monitoring set during training (no separate validation split), though no model selection was done on it (final-epoch models are saved).
- Timing is from one Apple M4 laptop, 1 thread, with other applications running; use it for relative comparison only. INT8 checkpoints are QNNPACK-specific and must be regenerated (ptq.py / qat.py) for an x86 FBGEMM deployment.
- QAT used a short schedule (3 x 20k images) because it must run on the CPU; a longer schedule or per-channel QAT weights may change the PTQ-vs-QAT ordering.
- Structured pruning prunes only the inner width of residual blocks; pruning the residual stream would give larger savings but needs coupled pruning across a whole stage.
- The 'screenshots' in the appendix are the saved console logs of each run rendered to images, not operating-system screen captures.

6. Deliverables map

Deliverable	Files
1 Baseline	train_baseline.py, checkpoints/resnet18_fp32.pth
2 PTQ	ptq.py, calibration.py, quant_utils.py, checkpoints/resnet18_ptq_int8(.pth, _ts.pt)
3 QAT	qat.py, checkpoints/resnet18_qat_int8(.pth, _ts.pt)
4 Unstructured pruning	prune_unstructured.py, checkpoints/resnet18_unstructured_pruned.pth
5 Structured pruning	prune_structured.py, checkpoints/resnet18_structured_pruned.pth
6 Inference pipeline	inference.py (all five models)
7 Benchmarking	benchmark.py, results/*.csv
8 ONNX	export_onnx.py, onnx_inference.py, onnx_models/*.onnx, results/onnx_benchmark.csv
9 Report	report/Technical_Report.pdf / .docx
10 README	README.md, requirements.txt, run_all.sh

Appendix - console output of each step

```
$ python train_baseline.py   [Baseline training]

======================================================================
epoch   1/20 | loss 1.3666 | train  50.53% | test  58.61% | lr 0.0181 | 113.9s
epoch   2/20 | loss 0.9054 | train  68.46% | test  67.61% | lr 0.0520 | 123.4s
epoch   3/20 | loss 0.7209 | train  75.08% | test  74.14% | lr 0.0860 | 134.1s
epoch   4/20 | loss 0.6084 | train  79.14% | test  75.36% | lr 0.1000 | 129.2s
epoch   5/20 | loss 0.5260 | train  82.00% | test  79.81% | lr 0.0990 | 132.9s
epoch   6/20 | loss 0.4729 | train  83.83% | test  78.73% | lr 0.0962 | 166.0s
epoch   7/20 | loss 0.4329 | train  85.01% | test  83.46% | lr 0.0916 | 146.6s
epoch   8/20 | loss 0.3993 | train  86.16% | test  84.52% | lr 0.0853 | 186.2s
epoch   9/20 | loss 0.3736 | train  87.31% | test  84.41% | lr 0.0778 | 166.0s
epoch  10/20 | loss 0.3482 | train  87.99% | test  84.47% | lr 0.0691 | 122.0s
epoch  11/20 | loss 0.3246 | train  88.81% | test  86.05% | lr 0.0597 | 152.9s
epoch  12/20 | loss 0.3038 | train  89.59% | test  86.88% | lr 0.0500 | 177.7s
epoch  13/20 | loss 0.2812 | train  90.31% | test  86.18% | lr 0.0402 | 161.1s
epoch  14/20 | loss 0.2535 | train  91.21% | test  87.46% | lr 0.0308 | 182.3s
epoch  15/20 | loss 0.2198 | train  92.53% | test  88.59% | lr 0.0222 | 285.4s
epoch  16/20 | loss 0.1882 | train  93.50% | test  89.82% | lr 0.0146 | 189.5s
epoch  17/20 | loss 0.1478 | train  95.03% | test  91.90% | lr 0.0084 | 192.6s
epoch  18/20 | loss 0.1036 | train  96.57% | test  92.77% | lr 0.0038 | 171.8s
epoch  19/20 | loss 0.0724 | train  97.68% | test  93.39% | lr 0.0010 | 115.0s
epoch  20/20 | loss 0.0589 | train  98.14% | test  93.43% | lr 0.0000 | 147.3s

Final FP32 test accuracy: 93.43%  (best epoch 93.43%)
Total training time: 53.4 min
Saved checkpoint -> /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_fp32.pth
Training log     -> /Users/amnahameed/Downloads/assignment3/logs/baseline_train_log.csv
```

A1 - baseline training (last epochs)

```
$ python ptq.py   [PTQ]


======================================================================
  PTQ step 1/6 - load FP32 model
======================================================================
FP32 test accuracy: 93.43%  (44.77 MB)

======================================================================
  PTQ step 2-3/6 - fuse layers + configure backend
======================================================================
quantisation backend : qnnpack   (supported: ['qnnpack', 'none'])
weights              : per-channel INT8 (symmetric)
activations          : UINT8 (histogram observer)
fused modules        : 9 ConvReLU2d + conv-only (after BN folding)
observers inserted   : 30

======================================================================
  PTQ step 4-5/6 - calibration on training subset
======================================================================
  calibration: 500/2000 images
  calibration: 1000/2000 images
  calibration: 1500/2000 images
  calibration: 2000/2000 images
calibrated with 2000 training images

======================================================================
  PTQ step 6/6 - convert to INT8, evaluate, save
======================================================================
INT8 (PTQ) test accuracy : 93.53%   (FP32 93.43%, drop -0.10 pts)
state_dict checkpoint    : /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_ptq_int8.pth  (11.31 MB)
TorchScript checkpoint   : /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_ptq_int8_ts.pt  (11.31 MB)
reloaded TorchScript acc : 93.53%  (OK)
```

A2 - PTQ

```
$ python qat.py   [QAT]


======================================================================
  QAT step 1/6 - load pretrained FP32 model
======================================================================
FP32 test accuracy: 93.43%

======================================================================
  QAT step 2-3/6 - fuse (train mode) + insert FakeQuantize modules
======================================================================
backend: qnnpack | fake-quant modules inserted: 51 | training device: cpu
accuracy right after prepare_qat (uncalibrated observers): 93.39%

======================================================================
  QAT step 4/6 - fine-tuning with fake quantisation
======================================================================
epoch 1/3 | loss 0.0588 | train 98.03% | fake-quant test 93.23% | 1017s
epoch 2/3 | loss 0.0572 | train 98.07% | fake-quant test 93.26% | 884s
epoch 3/3 | loss 0.0466 | train 98.40% | fake-quant test 93.20% | 768s

======================================================================
  QAT step 5-6/6 - convert to INT8, evaluate, save
======================================================================
INT8 (QAT) test accuracy : 93.24%   (FP32 93.43%, drop +0.19 pts)
state_dict checkpoint    : /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_qat_int8.pth  (11.22 MB)
TorchScript checkpoint   : /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_qat_int8_ts.pt  (11.23 MB)
fake-quant checkpoint    : /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_qat_fakequant.pth  (for ONNX QDQ export)
reloaded TorchScript acc : 93.24%  (OK)
```

A3 - QAT

```
$ python unstructured.py   [Unstructured pruning]

    [iter3] epoch 1/1 | loss 0.0798 | train 98.05% | test 93.21%

--- iteration 4/5: target 89.3% | achieved 89.28% | accuracy right after pruning 74.43%
    [iter4] epoch 1/1 | loss 0.0774 | train 97.75% | test 93.21%

--- iteration 5/5: target 90.0% | achieved 90.00% | accuracy right after pruning 90.73%
    [iter5] epoch 1/1 | loss 0.0678 | train 97.94% | test 93.01%

--- final fine-tuning (6 epochs, mask kept) ---
    [final] epoch 1/6 | loss 0.0694 | train 97.83% | test 92.43%
    [final] epoch 2/6 | loss 0.0616 | train 98.03% | test 92.86%
    [final] epoch 3/6 | loss 0.0487 | train 98.57% | test 93.22%
    [final] epoch 4/6 | loss 0.0365 | train 98.96% | test 93.41%
    [final] epoch 5/6 | loss 0.0296 | train 99.19% | test 93.51%
    [final] epoch 6/6 | loss 0.0262 | train 99.35% | test 93.47%

================================================================================
  Making pruning permanent (prune.remove)
================================================================================
sparsity before remove: 90.0000% | after remove: 90.0000%
accuracy after remove : 93.47%  (must equal 93.47%)

Per-layer sparsity:
  conv1                     (64, 3, 3, 3)          sparsity   9.5%
  layer1.0.conv1            (64, 64, 3, 3)         sparsity  45.3%
  layer1.0.conv2            (64, 64, 3, 3)         sparsity  40.1%
  layer1.1.conv1            (64, 64, 3, 3)         sparsity  40.7%
  layer1.1.conv2            (64, 64, 3, 3)         sparsity  41.1%
  layer2.0.conv1            (128, 64, 3, 3)        sparsity  46.2%
  layer2.0.conv2            (128, 128, 3, 3)       sparsity  51.2%
  layer2.0.shortcut.0       (128, 64, 1, 1)        sparsity  23.4%
  layer2.1.conv1            (128, 128, 3, 3)       sparsity  52.5%
  layer2.1.conv2            (128, 128, 3, 3)       sparsity  56.6%
  layer3.0.conv1            (256, 128, 3, 3)       sparsity  60.6%
  layer3.0.conv2            (256, 256, 3, 3)       sparsity  68.4%
  layer3.0.shortcut.0       (256, 128, 1, 1)       sparsity  34.1%
  layer3.1.conv1            (256, 256, 3, 3)       sparsity  70.4%
  layer3.1.conv2            (256, 256, 3, 3)       sparsity  80.8%
  layer4.0.conv1            (512, 256, 3, 3)       sparsity  93.3%
  layer4.0.conv2            (512, 512, 3, 3)       sparsity  99.1%
  layer4.0.shortcut.0       (512, 256, 1, 1)       sparsity  64.5%
  layer4.1.conv1            (512, 512, 3, 3)       sparsity  99.6%
  layer4.1.conv2            (512, 512, 3, 3)       sparsity  99.9%
  fc                        (10, 512)              sparsity   8.0%

non-zero weights: 1,116,436 / 11,164,352  (90.00% sparse)
accuracy: baseline 93.43% -> pruned 93.47%  (+0.04 pts)
saved -> /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_unstructured_pruned.pth  (44.77 MB raw, 7.09 MB gzip)
```

A4 - unstructured pruning

```
$ python structured.py   [Structured pruning]


================================================================================
  Structured channel pruning | device=mps | baseline acc 93.43%
================================================================================
block        mid channels   kept  removed
layer1.0              64     48       16   (ratio 25%)
layer1.1              64     48       16   (ratio 25%)
layer2.0             128     77       51   (ratio 40%)
layer2.1             128     77       51   (ratio 40%)
layer3.0             256    102      154   (ratio 60%)
layer3.1             256    102      154   (ratio 60%)
layer4.0             512    154      358   (ratio 70%)
layer4.1             512    154      358   (ratio 70%)

reconstruction check: max |logit diff| vs masked original = 0.00e+00  (OK)
new internal widths: [48, 48, 77, 77, 102, 102, 154, 154]
accuracy right after pruning (no fine-tuning): 30.20%

================================================================================
  Fine-tuning pruned network (12 epochs)
================================================================================
    [structured] epoch 1/12 | loss 0.2648 | train 90.84% | test 87.33%
    [structured] epoch 2/12 | loss 0.2187 | train 92.43% | test 88.50%
    [structured] epoch 3/12 | loss 0.1966 | train 93.16% | test 88.77%
    [structured] epoch 4/12 | loss 0.1763 | train 93.95% | test 88.24%
    [structured] epoch 5/12 | loss 0.1490 | train 94.91% | test 90.52%
    [structured] epoch 6/12 | loss 0.1233 | train 95.80% | test 91.73%
    [structured] epoch 7/12 | loss 0.0997 | train 96.61% | test 92.17%
    [structured] epoch 8/12 | loss 0.0757 | train 97.55% | test 92.49%
    [structured] epoch 9/12 | loss 0.0547 | train 98.14% | test 93.11%
    [structured] epoch 10/12 | loss 0.0424 | train 98.70% | test 93.44%
    [structured] epoch 11/12 | loss 0.0342 | train 99.04% | test 93.48%
    [structured] epoch 12/12 | loss 0.0328 | train 99.07% | test 93.34%

zero-valued conv1 filters left in rebuilt model: 0 (no masked channels)
parameters: 11,173,962 -> 3,913,470  (65.0% fewer)
MACs      : 555.4M -> 293.2M  (47.2% fewer)
accuracy  : 93.43% -> 30.20% (pruned) -> 93.34% (fine-tuned)
saved -> /Users/amnahameed/Downloads/assignment3/checkpoints/resnet18_structured_pruned.pth  (15.72 MB)
```

A5 - structured pruning

```
$ python export_onnx.py   [ONNX export]


================================================================================
  Export FP32 / pruned float models
================================================================================
fp32         -> resnet18_fp32.onnx  (44.69 MB)  [onnx checker OK]
unstructured  -> resnet18_unstructured_pruned.onnx  (44.69 MB)  [onnx checker OK]
structured    -> resnet18_structured_pruned.onnx  (15.65 MB)  [onnx checker OK]

================================================================================
  Export PTQ-INT8 (QDQ re-expression of the converted INT8 model)
================================================================================
PyTorch INT8 accuracy (rebuilt from state_dict): 93.53%
QDQ twin vs INT8 model on 500 images: argmax agreement 99.6%, mean |logit diff| 0.0171
ptq          -> resnet18_ptq_int8.onnx  (11.24 MB)  [onnx checker OK]
  QuantizeLinear: 31 | DequantizeLinear: 60 | Conv: 20 | Gemm/MatMul: 1

================================================================================
  Export QAT-INT8 (QDQ re-expression of the converted INT8 model)
================================================================================
PyTorch INT8 accuracy (rebuilt from state_dict): 93.24%
QDQ twin vs INT8 model on 500 images: argmax agreement 99.8%, mean |logit diff| 0.0293
qat          -> resnet18_qat_int8.onnx  (11.22 MB)  [onnx checker OK]
  QuantizeLinear: 31 | DequantizeLinear: 60 | Conv: 20 | Gemm/MatMul: 1
```

A6 - ONNX export

```
$ python onnx_benchmark.py   [ONNX benchmark]

================================================================================
  ONNX Runtime (CPU, 1 thread): Structured pruned  [resnet18_structured_pruned.onnx]
================================================================================
accuracy 93.34% | agreement with FP32 ONNX 95.53% | latency 6.14 ms | throughput 166 img/s | 15.65 MB

================================================================================
  ONNX SUMMARY
================================================================================
model accuracy_pct agreement_with_fp32_pct size_mb latency_bs1_mean_ms latency_bs1_p50_ms latency_bs1_p95_ms throughput_img_s onnx_file ...
FP32 baseline 93.43 100.00 44.69 11.290 11.274 11.409 92.7 resnet18_fp32.onnx 1.00
PTQ INT8 (QDQ) 93.42 99.66 11.24 12.048 11.896 12.642 91.1 resnet18_ptq_int8.onnx 0.94
QAT INT8 (QDQ) 93.27 97.88 11.22 12.016 11.821 11.970 90.2 resnet18_qat_int8.onnx 0.94
Unstructured pruned 93.47 96.78 44.69 11.281 11.270 11.409 92.8 resnet18_unstructured_pruned.onnx 1.00
Structured pruned 93.34 95.53 15.65 6.140 6.130 6.241 166.4 resnet18_structured_pruned.onnx 1.84

written: /Users/amnahameed/Downloads/assignment3/results/onnx_benchmark.csv
```

A7 - ONNX-Runtime benchmark